Universidad Autónoma de Baja California

FACULTAD DE INGENIERÍA MEXICALI

Ingeniería en Computación

6to Semestre

Materia:

Microcontroladores

Actividad:

Práctica 4 Calculadora LCD

Profesor:

Carlos Alberto Aguilar Avelar

Alumnos:

Benitez López Cesar Alfonso

Mayorga Meza Jesus Alexis

20 de Marzo del 2026, Mexicali, Baja California

Introducción **3**
Microcontrolador ATmega328P 3
Pantalla LCD 16×2 3
Teclado Matricial 4×4 3
Interrupciones por cambio de pin (PCINT) 3
Aritmética de punto flotante en microcontroladores 3
Descripción **4**
Desarrollo **4**
Asignación de teclas 5
Código fuente del firmware 5
Descripción del algoritmo 5
Simulación en Proteus 6
Implementación en laboratorio 9
Conclusiones **10**
Referencias **10**

Introducción

Microcontrolador ATmega328P

El ATmega328P es un microcontrolador de 8 bits de la familia AVR fabricado por Microchip Technology (anteriormente Atmel). Opera a frecuencias de hasta 20 MHz, cuenta con 32 KB de memoria Flash, 2 KB de SRAM y 1 KB de EEPROM. Dispone de 23 líneas de I/O de propósito general organizadas en tres puertos (B, C y D), múltiples módulos de temporización, UART, SPI, I²C y conversor ADC de 10 bits. Gracias a estas características, es ampliamente utilizado en proyectos de sistemas embebidos y educativos.

Pantalla LCD 16×2

Una pantalla LCD (Liquid Crystal Display) 16×2 puede mostrar hasta 16 caracteres por fila en 2 filas, utilizando el controlador estándar HD44780 o compatible. La comunicación con el microcontrolador puede realizarse en modo de 8 bits (usando 8 líneas de datos) o en modo de 4 bits (usando únicamente D4–D7), lo que ahorra pines de I/O. Requiere tres líneas de control: RS (Register Select), RW (Read/Write) y EN (Enable). En esta práctica se empleó el modo de 4 bits conectado al PORTD del ATmega328P.

Teclado Matricial 4×4

Un teclado matricial de 4×4 contiene 16 teclas organizadas en una cuadrícula de 4 filas y 4 columnas. Para detectar una pulsación, se activa una fila a la vez llevándola a nivel bajo, mientras el resto permanece en nivel alto; si alguna columna también registra nivel bajo, se identifica la tecla presionada por la intersección fila-columna. Este mecanismo reduce el número de pines necesarios de 16 a sólo 8 (4 filas + 4 columnas).

Interrupciones por cambio de pin (PCINT)

El ATmega328P soporta interrupciones de cambio de pin (Pin Change Interrupt, PCINT) que permiten detectar transiciones lógicas en cualquier pin de los puertos B, C y D sin necesidad de sondear continuamente (polling). Se configuran habilitando el bit correspondiente en PCICR (Pin Change Interrupt Control Register) y la máscara en PCMSK0/PCMSK1/PCMSK2. En esta práctica, las cuatro filas del teclado se conectaron a PB0–PB3 y se habilitó la interrupción PCINT0_vect para detectar pulsaciones de forma eficiente.

Aritmética de punto flotante en microcontroladores

Los microcontroladores de 8 bits como el ATmega328P no cuentan con una unidad de punto flotante (FPU) por hardware; sin embargo, la biblioteca estándar de avr-libc provee soporte para el tipo double en software, permitiendo realizar operaciones aritméticas con decimales a costa de mayor tiempo de ejecución y espacio en Flash. En esta práctica se empleó el tipo double para soportar operandos de hasta 5 dígitos y resultados con hasta 2 dígitos de precisión decimal.

Descripción

Programar el firmware de una calculadora básica utilizando una pantalla LCD 16x2 y un teclado matricial de 4x4.

El funcionamiento de la calculadora debe ser el siguiente:

- La tecla A es la operación de suma (+)
- La tecla B es la operación de resta (-)
- La tecla C es la operación de multiplicación (x)
- La tecla D es la operación de división (/)
- La tecla # es la tecla igual (=)
- La tecla * es la tecla borrar la pantalla y memoria (C)

Especificaciones para el desarrollo:

- Debe realizar operaciones con números hasta 5 dígitos (99,999x99,999=9,999,800,001).
- Debe permitir hacer operaciones con la respuesta de la operación anterior (Ansx2=19,999,600,002).
- Debe mostrar resultados con punto decimal y 2 dígitos de precisión.
- El teclado debe estar conectado directamente a un puerto del microcontrolador y debe ser monitoreado usando la Pin Change Interrupt.
- Debe indicar Error cuando se ingresen secuencias de dígitos inválidos o si el resultado no puede ser mostrado en la pantalla.
- El firmware no debe presentar bugs o comportamientos inesperados.

Desarrollo

El sistema implementa una calculadora de cuatro operaciones básicas (suma, resta, multiplicación y división) utilizando un ATmega328P como unidad de procesamiento central. La interfaz de entrada es un teclado matricial 4×4 monitorizado mediante la interrupción PCINT, y la interfaz de salida es un LCD 16×2 en modo 4 bits. El sistema soporta operandos de hasta 5 dígitos (0–99,999), resultados de hasta 10 dígitos con 2 decimales, operaciones encadenadas usando el resultado anterior (ANS), y detección de errores como división por cero o desbordamiento de pantalla.

Asignación de teclas

Tecla	Función	Descripción
0 – 9	Dígitos numéricos	Ingreso de operandos
A	Suma (+)	Inicia operación de adición
B	Resta (−)	Inicia operación de sustracción
C	Multiplicación (×)	Inicia operación de multiplicación
D	División (÷)	Inicia operación de división
#	Igual (=)	Calcula y muestra el resultado
*	Borrar (C)	Limpia pantalla y memoria

Código fuente del firmware

El siguiente código fue desarrollado en lenguaje C para el compilador avr-gcc. Incluye comentarios explicativos en cada bloque funcional. El firmware implementa una máquina de estados para el procesamiento de teclas, manejo de la interrupción PCINT para el teclado y rutinas de control del LCD en modo 4 bits.

```C/C++
#define F_CPU 16000000
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <stdio.h>
#include "LCD.h"

typedef enum
{
```

```
        Esperandonum1 = 0,
        Esperando_ope,
        Esperandonum2,
        Mostrar_res,
        Error
} estado_calculadora_t;

static uint32_t num1 = 0;
static uint32_t num2 = 0;
static uint8_t digi1 = 0;
static uint8_t digi2 = 0;
static uint8_t entrada1 = 0;
static uint8_t entrada2 = 0;
static char opeactual = 0;
static int64_t ans_x100 = 0;
static uint8_t isanswer = 0;
static uint8_t use_ans = 0;
static int64_t val_izq_x100 = 0;

static estado_calculadora_t estado = Esperandonum1;

static void limpiar_linea(uint8_t linea);
static void mostrar_superior(const char *texto);
static void mostrar_inferior(const char *texto);
static void mostrar_error(const char *texto);
static void reiniciar_estado_sin_borrar_ans(void);
static void limpiar_calculadora_total(void);
static void agregar_digito(char tecla);
static void seleccionar_operador(char tecla);
static void calcular_resultado(void);
static void mostrar_expresion_actual(void);
static void formatear_x100(int64_t valor_x100, char *buffer);

static void formatear_x100(int64_t valor_x100, char *buffer)
{
        uint64_t abs_val;
        uint8_t is_neg = 0;

        if(valor_x100 < 0) {
                is_neg = 1;
                abs_val = (uint64_t)(-valor_x100);
        } else {
                abs_val = (uint64_t)valor_x100;
        }

        uint64_t parte_entera = abs_val / 100;
        uint8_t parte_decimal = abs_val % 100;
```

```
        char temp[24];
        int i = 0, j = 0;

        if(parte_entera == 0) {
                temp[i++] = '0';
        } else {
                while(parte_entera > 0) {
                        temp[i++] = (parte_entera % 10) + '0';
                        parte_entera /= 10;
                }
        }

        if(is_neg) temp[i++] = '-';

        while(i > 0) {
                buffer[j++] = temp[--i];
        }

        buffer[j++] = '.';
        buffer[j++] = (parte_decimal / 10) + '0';
        buffer[j++] = (parte_decimal % 10) + '0';
        buffer[j] = '\0';
}

static void limpiar_linea(uint8_t linea)
{
        lcd_goto_xy(linea, 0);
        lcd_write_word((uint8_t *)"                "); 
}

static void mostrar_superior(const char *texto)
{
        limpiar_linea(0);
        lcd_goto_xy(0, 0);
        lcd_write_word((uint8_t *)texto);
}

static void mostrar_inferior(const char *texto)
{
        limpiar_linea(1);
        lcd_goto_xy(1, 0);
        lcd_write_word((uint8_t *)texto);
}

static void mostrar_error(const char *texto)
{
```

```
	estado = Error;
	mostrar_superior("Error");
	mostrar_inferior(texto);
}

static void reiniciar_estado_sin_borrar_ans(void)
{
	num1 = 0;
	num2 = 0;
	digi1 = 0;
	digi2 = 0;
	entrada1 = 0;
	entrada2 = 0;
	opeactual = 0;
	use_ans = 0;
	val_izq_x100 = 0;
	estado = Esperandonum1;
}

static void limpiar_calculadora_total(void)
{
	reiniciar_estado_sin_borrar_ans();
	ans_x100 = 0;
	isanswer = 0;

	mostrar_superior("Practica 4");
	mostrar_inferior("Benitez-Mayorga");
}

void inicializar_calculadora(void)
{
	lcd_clear();
	_delay_ms(20);
	lcd_clear();
	_delay_ms(20);
	limpiar_calculadora_total();
}

void procesar_tecla(char tecla)
{
	if(estado == Error)
	{
		if(tecla == '*') {
			limpiar_calculadora_total();
			return;
		}
```

```
            if(tecla >= '0' && tecla <= '9') {
                reiniciar_estado_sin_borrar_ans();
            }
            else if(tecla == 'A' || tecla == 'B' || tecla == 'C' || tecla ==
'D') {
                if(!isanswer) {
                    mostrar_error("Asi no se usa");
                    return;
                }
                reiniciar_estado_sin_borrar_ans();
                use_ans = 1;
                val_izq_x100 = ans_x100;
                opeactual = tecla;
                estado = Esperandonum2;
                mostrar_expresion_actual();
                return;
            } else {
                return;
            }
        }

        if(tecla >= '0' && tecla <= '9') {
            agregar_digito(tecla);
        }
        else if(tecla == 'A' || tecla == 'B' || tecla == 'C' || tecla == 'D') {
            seleccionar_operador(tecla);
        }
        else if(tecla == '#') {
            calcular_resultado();
        }
        else if(tecla == '*') {
            limpiar_calculadora_total();
        }
}

static void mostrar_expresion_actual(void)
{
        char lin0[17];
        char lin1[17];
        uint8_t idx = 0;
        char op_char = ' ';
        char buf[12];

        memset(lin0, 0, sizeof(lin0));
        memset(lin1, 0, sizeof(lin1));

        if(use_ans) {
```

```
                lin0[idx++] = 'A';
                lin0[idx++] = 'n';
                lin0[idx++] = 's';
        } else if(entrada1) {
                sprintf(buf, "%lu", num1);
                for(uint8_t i = 0; buf[i] != '\0' && idx < 16; i++)
                        lin0[idx++] = buf[i];
        }

        switch(opeactual) {
                case 'A': op_char = '+'; break;
                case 'B': op_char = '-'; break;
                case 'C': op_char = 'x'; break;
                case 'D': op_char = '/'; break;
                default:  op_char = ' '; break;
        }

        if(opeactual != 0 && idx < 16)
                lin0[idx++] = op_char;

        if(entrada2) {
                sprintf(buf, "%lu", num2);
                for(uint8_t i = 0; buf[i] != '\0' && idx < 16; i++)
                        lin0[idx++] = buf[i];
        }

        if(estado == Esperandonum1) {
                strcpy(lin1, "Ingresa un numero");
        } else if(estado == Esperando_ope) {
                strcpy(lin1, "Ingresa operador");
        } else if(estado == Esperandonum2) {
                strcpy(lin1, "Ingresa otro num");
        } else if(estado == Mostrar_res) {
                strcpy(lin1, "Ans disponible");
        }

        mostrar_superior(lin0);
        mostrar_inferior(lin1);
}

static void agregar_digito(char tecla)
{
        if(estado == Mostrar_res) {
                reiniciar_estado_sin_borrar_ans();
        }

        if(estado == Esperandonum1 || estado == Esperando_ope) {
```

```
		if(digi1 >= 5) {
			mostrar_error("Max 5 digitos");
			return;
		}
		num1 = (num1 * 10UL) + (uint32_t)(tecla - '0');
		digi1++;
		entrada1 = 1;
		estado = Esperando_ope;
		mostrar_expresion_actual();
		return;
	}

	if(estado == Esperandonum2) {
		if(digi2 >= 5) {
			mostrar_error("Max 5 digitos");
			return;
		}
		num2 = (num2 * 10UL) + (uint32_t)(tecla - '0');
		digi2++;
		entrada2 = 1;
		mostrar_expresion_actual();
		return;
	}
}

static void seleccionar_operador(char tecla)
{
	if(estado == Mostrar_res) {
		if(!isanswer) {
			mostrar_error("Sin Ans");
			return;
		}
		reiniciar_estado_sin_borrar_ans();
		use_ans = 1;
		val_izq_x100 = ans_x100;
		opeactual = tecla;
		estado = Esperandonum2;
		mostrar_expresion_actual();
		return;
	}

	if(estado == Esperandonum1) {
		if(!isanswer) {
			mostrar_error("Falta operando");
			return;
		}
		use_ans = 1;
```

```
                val_izq_x100 = ans_x100;
                opeactual = tecla;
                estado = Esperandonum2;
                mostrar_expresion_actual();
                return;
        }

        if(estado == Esperando_ope) {
                if(!entrada1) {
                        mostrar_error("Falta operando");
                        return;
                }
                opeactual = tecla;
                estado = Esperandonum2;
                mostrar_expresion_actual();
                return;
        }

        if(estado == Esperandonum2) {
                mostrar_error("Op duplicada");
                return;
        }
}

static void calcular_resultado(void)
{
        char txt_res[24];
        int64_t izq_x100 = 0;
        int64_t der_raw = 0;
        int64_t der_x100 = 0;
        int64_t res_x100 = 0;

        if(estado != Esperandonum2) {
                mostrar_error("Wrong Anss");
                return;
        }
        if(opeactual == 0) {
                mostrar_error("Sin operacion");
                return;
        }
        if(!entrada2) {
                mostrar_error("Falta el 2do numero");
                return;
        }

        if(use_ans) {
                izq_x100 = val_izq_x100;
```

```
} else {
        izq_x100 = (int64_t)num1 * 100;
}

der_raw = (int64_t)num2;
der_x100 = der_raw * 100;

switch(opeactual)
{
        case 'A': // Suma
                res_x100 = izq_x100 + der_x100;
                break;
        case 'B': // Resta
                res_x100 = izq_x100 - der_x100;
                break;
        case 'C': // Multiplicación
                res_x100 = izq_x100 * der_raw;
                break;
        case 'D': // División
                if(der_raw == 0) {
                        mostrar_error("Division por 0");
                        return;
                }
                res_x100 = izq_x100 / der_raw;
                break;
}

formatear_x100(res_x100, txt_res);

if(strlen(txt_res) > 16) {
        mostrar_error("No cabe");
        return;
}

mostrar_superior("Answer:");
mostrar_inferior(txt_res);

ans_x100 = res_x100;
isanswer = 1;
num1 = 0;
num2 = 0;
digi1 = 0;
digi2 = 0;
entrada1 = 0;
entrada2 = 0;
opeactual = 0;
val_izq_x100 = 0;
```

```
        use_ans = 0;
        estado = Mostrar_res;
}
```

```C/C++
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdint.h>
#include "LCD.h"

void inicializar_calculadora(void);
void procesar_tecla(char tecla);

#define COL_PORT   PORTD
#define COL_DDR    DDRD
#define COL_PIN    PIND
#define COL0 PD2
#define COL1 PD3
#define COL2 PD4
#define COL3 PD5
#define ROW_PORTD  PORTD
#define ROW_DDRD   DDRD
#define ROW_PORTB  PORTB
#define ROW_DDRB   DDRB
#define ROW0 PD6
#define ROW1 PD7
#define ROW2 PB0
#define ROW3 PB1

volatile uint8_t flag_tecla = 0;

static void filas_en_bajo(void)
{
        ROW_PORTD &= ~((1 << ROW0) | (1 << ROW1));
        ROW_PORTB &= ~((1 << ROW2) | (1 << ROW3));
}

static void filas_en_alto(void)
{
        ROW_PORTD |= (1 << ROW0) | (1 << ROW1);
```

```
        ROW_PORTB |= (1 << ROW2) | (1 << ROW3);
}

static void keypad_init(void)
{
        COL_DDR &= ~((1 << COL0) | (1 << COL1) | (1 << COL2) | (1 << COL3));
        COL_PORT |= (1 << COL0) | (1 << COL1) | (1 << COL2) | (1 << COL3);

        ROW_DDRD |= (1 << ROW0) | (1 << ROW1);
        ROW_DDRB |= (1 << ROW2) | (1 << ROW3);

        filas_en_bajo();

        PCICR |= (1 << PCIE2);
        PCMSK2 |= (1 << PCINT18) | (1 << PCINT19) | (1 << PCINT20) | (1 <<
PCINT21);
}

static char keypad_scan(void)
{
        const char keymap[4][4] = {
                {'1','2','3','A'},
                {'4','5','6','B'},
                {'7','8','9','C'},
                {'*','0','#','D'}
        };

        for(uint8_t r = 0; r < 4; r++)
        {
                filas_en_alto();

                switch(r)
                {
                        case 0: ROW_PORTD &= ~(1 << ROW0); break;
                        case 1: ROW_PORTD &= ~(1 << ROW1); break;
                        case 2: ROW_PORTB &= ~(1 << ROW2); break;
                        case 3: ROW_PORTB &= ~(1 << ROW3); break;
                }

                _delay_us(10);

                if(!(COL_PIN & (1 << COL0)))
                {
                        while(!(COL_PIN & (1 << COL0)));
                        filas_en_bajo();
                        return keymap[r][0];
                }
```

```
            if(!(COL_PIN & (1 << COL1)))
            {
                  while(!(COL_PIN & (1 << COL1)));
                  filas_en_bajo();
                  return keymap[r][1];
            }

            if(!(COL_PIN & (1 << COL2)))
            {
                  while(!(COL_PIN & (1 << COL2)));
                  filas_en_bajo();
                  return keymap[r][2];
            }

            if(!(COL_PIN & (1 << COL3)))
            {
                  while(!(COL_PIN & (1 << COL3)));
                  filas_en_bajo();
                  return keymap[r][3];
            }
      }

      filas_en_bajo();
      return 0;
}

ISR(PCINT2_vect)
{
      flag_tecla = 1;
}

int main(void)
{
      lcd_init();
      keypad_init();
      inicializar_calculadora();

      sei();

      while(1)
      {
            if(flag_tecla)
            {
                  _delay_ms(25);
                  char tecla = keypad_scan();
                  flag_tecla = 0;
```

```
                    if(tecla != 0)
                    {
                          procesar_tecla(tecla);
                    }
              }
        }
}
```

```C/C++
#ifndef LCD_H_
#define LCD_H_
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

#define DATA_BUS    PORTC
#define CTL_BUS     PORTC
#define DATA_DDR    DDRC
#define CTL_DDR     DDRC

#define LCD_RS      PC0
#define LCD_EN      PC1
#define LCD_D4      PC2
#define LCD_D5      PC3
#define LCD_D6      PC4
#define LCD_D7      PC5

#define LCD_CMD_CLEAR_DISPLAY            0x01
#define LCD_CMD_CURSOR_HOME              0x02

#define LCD_CMD_DISPLAY_OFF              0x08
#define LCD_CMD_DISPLAY_NO_CURSOR        0x0C
#define LCD_CMD_DISPLAY_CURSOR_NO_BLINK  0x0E
#define LCD_CMD_DISPLAY_CURSOR_BLINK     0x0F

#define LCD_CMD_4BIT_2ROW_5X7            0x28
#define LCD_CMD_8BIT_2ROW_5X7            0x38

void lcd_init(void);
void lcd_send_command(uint8_t command);
void lcd_write_character(uint8_t character);
```

```
void lcd_write_word(uint8_t word[]);
void lcd_clear(void);
void lcd_goto_xy(uint8_t line, uint8_t pos);

#endif /* LCD_H_ */
```

```
C/C++
#include "LCD.h"

void lcd_init(void)
{
	DATA_DDR |= (1 << LCD_D7) | (1 << LCD_D6) | (1 << LCD_D5) | (1 <<
LCD_D4);
	CTL_DDR  |= (1 << LCD_EN) | (1 << LCD_RS);

	DATA_BUS &= ~((1 << LCD_D7) | (1 << LCD_D6) | (1 << LCD_D5) | (1 <<
LCD_D4));
	CTL_BUS  &= ~(1 << LCD_RS);
	CTL_BUS  &= ~(1 << LCD_EN);

	_delay_ms(40);

	lcd_send_command(LCD_CMD_4BIT_2ROW_5X7);
	_delay_ms(2);

	lcd_send_command(LCD_CMD_DISPLAY_NO_CURSOR);
	_delay_ms(2);

	lcd_send_command(LCD_CMD_CLEAR_DISPLAY);
	_delay_ms(5);

	lcd_send_command(0x80);
	_delay_ms(2);
}

void lcd_send_command(uint8_t command)
{
	/* nibble alto */
	DATA_BUS |= ((command >> 2) & 0b00111100);
	DATA_BUS &= ~(((~(command >> 2)) & 0b00111100));

	CTL_BUS &= ~(1 << LCD_RS);
```

```
        CTL_BUS |=  (1 << LCD_EN);
        _delay_ms(1);
        CTL_BUS &= ~(1 << LCD_EN);
        _delay_ms(1);

        /* nibble bajo */
        DATA_BUS |= ((command << 2) & 0b00111100);
        DATA_BUS &= ~(((~(command << 2)) & 0b00111100));

        CTL_BUS &= ~(1 << LCD_RS);
        CTL_BUS |=  (1 << LCD_EN);
        _delay_ms(1);
        CTL_BUS &= ~(1 << LCD_EN);
        _delay_ms(1);
}

void lcd_write_character(uint8_t character)
{
        /* nibble alto */
        DATA_BUS |= ((character >> 2) & 0b00111100);
        DATA_BUS &= ~(((~(character >> 2)) & 0b00111100));

        CTL_BUS |=  (1 << LCD_RS);
        CTL_BUS |=  (1 << LCD_EN);
        _delay_ms(1);
        CTL_BUS &= ~(1 << LCD_EN);
        _delay_ms(1);

        /* nibble bajo */
        DATA_BUS |= ((character << 2) & 0b00111100);
        DATA_BUS &= ~(((~(character << 2)) & 0b00111100));

        CTL_BUS |=  (1 << LCD_RS);
        CTL_BUS |=  (1 << LCD_EN);
        _delay_ms(1);
        CTL_BUS &= ~(1 << LCD_EN);
        _delay_ms(1);
}

void lcd_write_word(uint8_t word[])
{
        uint8_t i = 0;

        while(word[i] != '\0')
        {
                lcd_write_character(word[i]);
                i++;
```

```
        }
}

void lcd_clear(void)
{
        lcd_send_command(LCD_CMD_CLEAR_DISPLAY);
        _delay_ms(5);
}

void lcd_goto_xy(uint8_t line, uint8_t pos)
{
        lcd_send_command((0x80 | (line << 6)) + pos);
        _delay_us(50);
}
```

Descripción del algoritmo

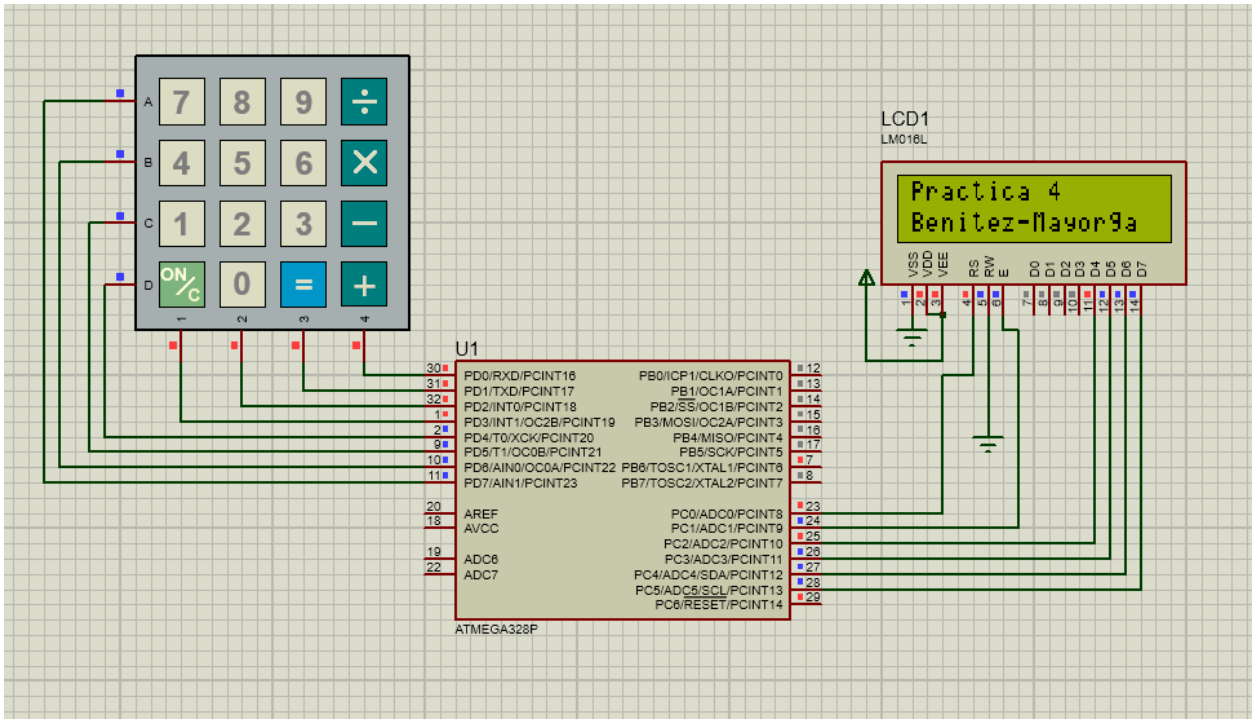
El firmware opera bajo el siguiente flujo de estados:

- Estado IDLE (reposo): El sistema espera una pulsación detectada por la interrupción PCINT0_vect. Al detectar un cambio en los pines PB0–PB3, se ejecuta un escaneo completo de la matriz para identificar la tecla presionada.
- Ingreso de dígitos: Cada dígito numérico se concatena en input_buf (máximo 5 caracteres). Se muestra en tiempo real en la segunda fila del LCD.
- Ingreso de operador (A/B/C/D): Se guarda el primer operando y el operador. Si existe un resultado previo (ANS), éste se usa automáticamente como primer operando.
- Ingreso del segundo operando: El usuario escribe el segundo número (hasta 5 dígitos).
- Cálculo (tecla #): Se evalúa la operación, se verifica el rango del resultado y se muestra con 2 decimales de precisión. El resultado queda en memoria como ANS para operaciones encadenadas.
- Borrado (*): Reinicia todas las variables y limpia la pantalla.
- Error: Ante división por cero, resultado fuera de rango o secuencia inválida, se muestra «ERROR» durante 2 segundos y se reinicia el sistema.

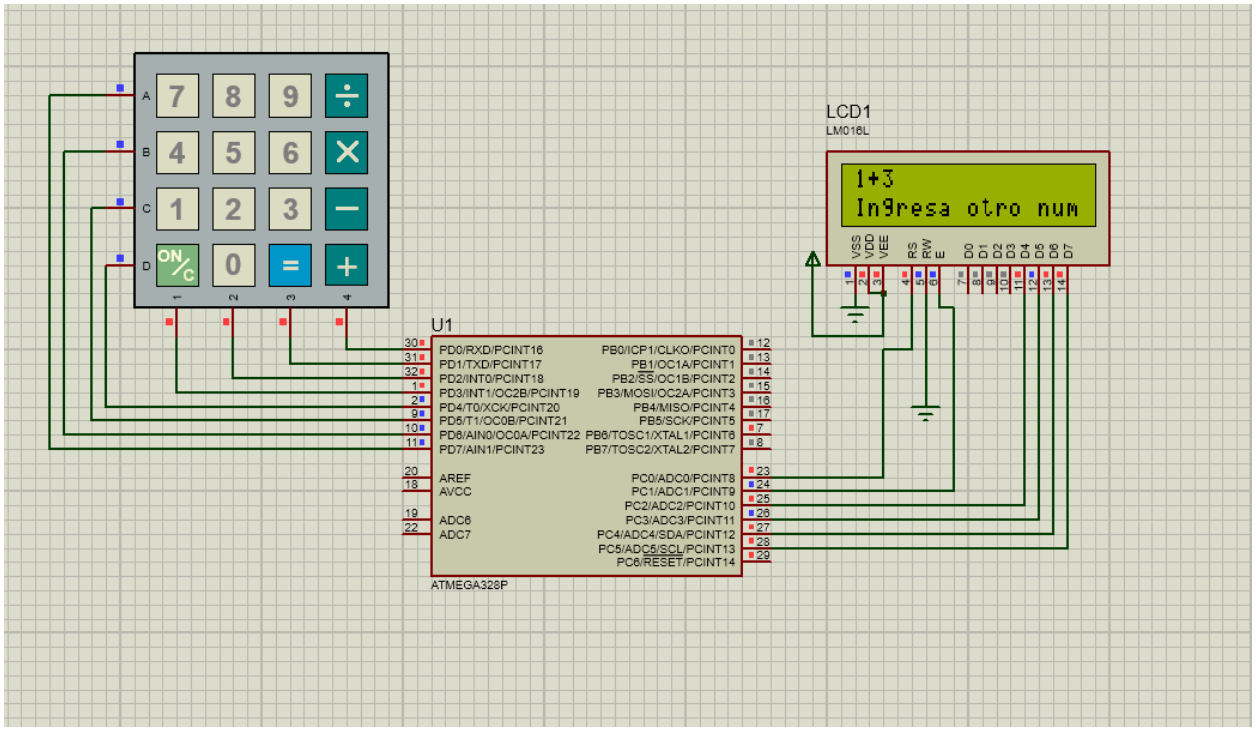
Simulación en Proteus

Se realizó la simulación del circuito completo en Proteus ISIS utilizando el modelo virtual del ATmega328P, un componente LCD 16×2 (LM016L) y un teclado matricial 4×4. La simulación permitió verificar el correcto funcionamiento de las rutinas de inicialización del LCD, la respuesta a pulsaciones del teclado y el cálculo de operaciones.

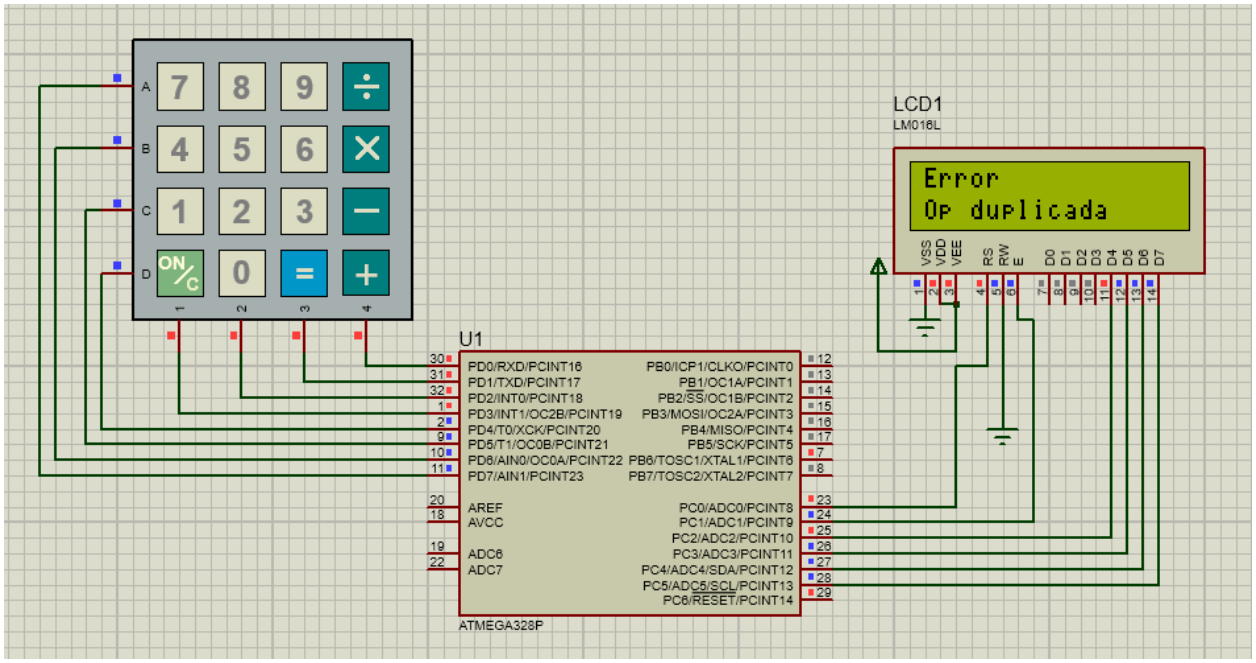
Prueba inicial:



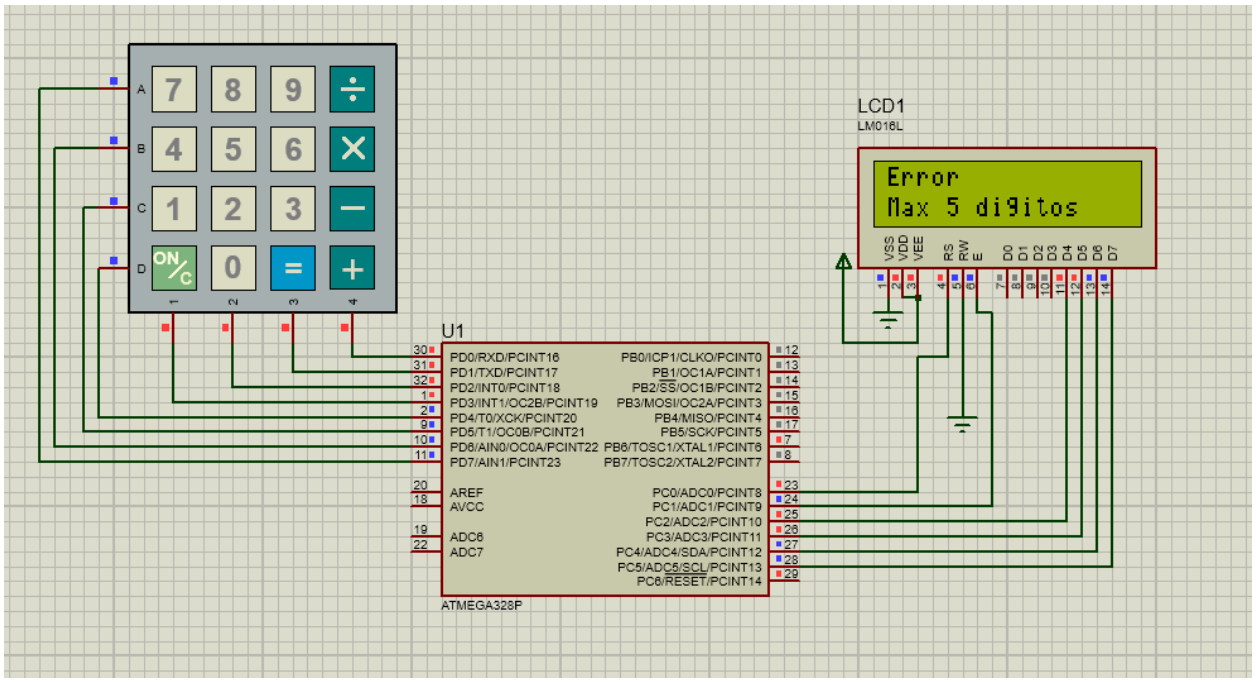
Operandos y Operadores:



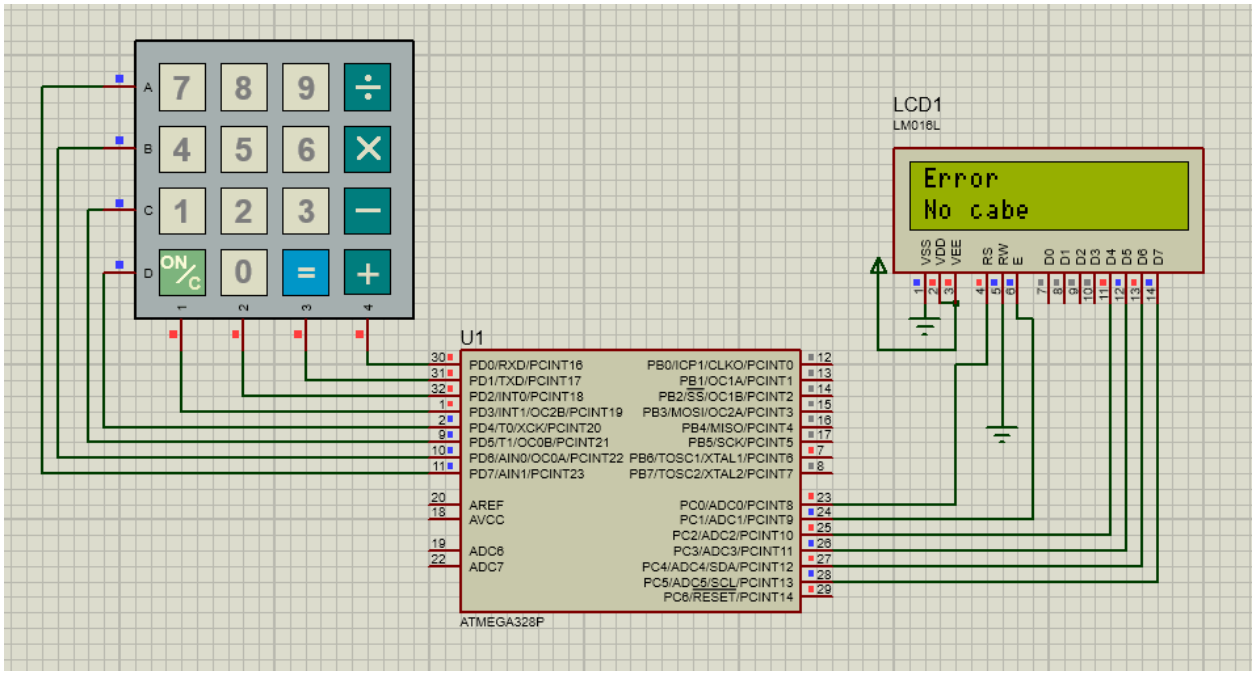
Error en caso de no marcar igual y seleccionar otro operando:



Error superando los 5 dígitos disponibles:



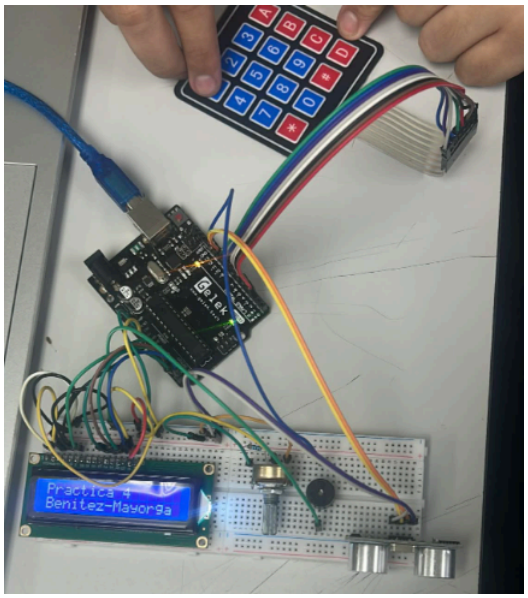
Error por Overflow:

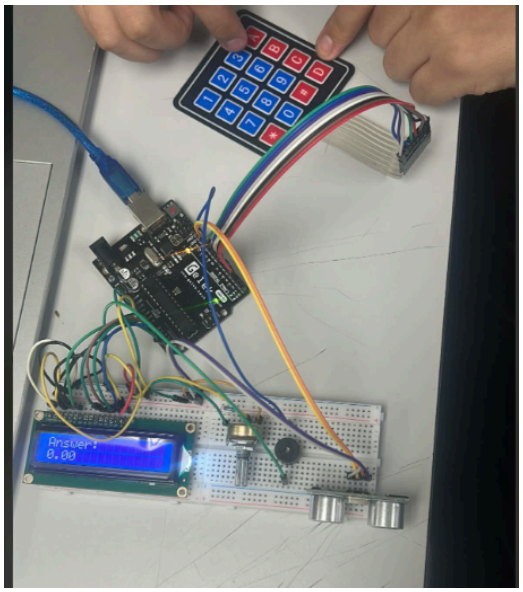


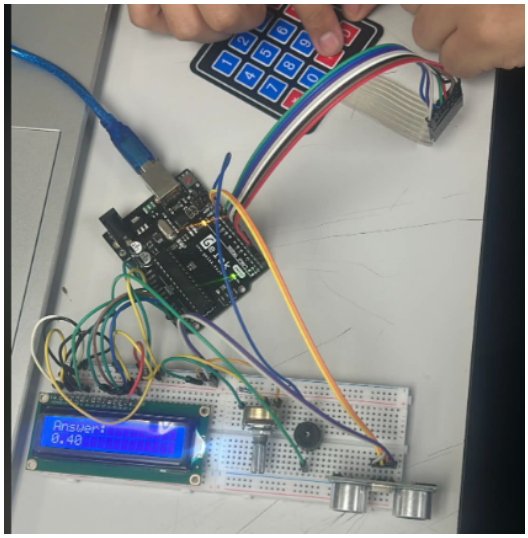
Ejemplo De Simulacion Completa:

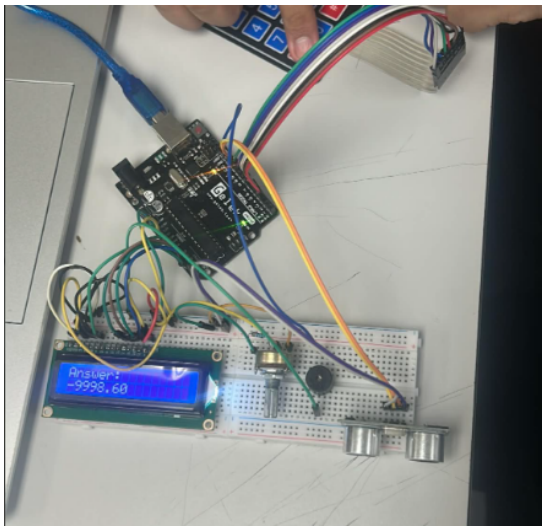
https://drive.google.com/file/d/16iVdcdlluD5vjlYOmZ-saclguwtqj7ev/view?usp=sharing

Implementación en laboratorio









Video de Presentacion:

https://drive.google.com/file/d/1dWw926YiZewthzTUgbv61_v6FJ2IpDmf/view?usp=sharing

Conclusiones

Benitez Lopez Cesar:

Realizando esta práctica llegue a la conclusión que no se podía realizar con float ya que solo representaba correctamente 7 bits después de eso los ultimos 3 digitos se generaban de basura; al hacer la práctica con el cálculo x100 funciono correctamente porque eliminamos los puntos flotantes

Mayorga Meza Alexis:

Con esta práctica aprendimos que es muy complejo el manejas pantallas LCD, cuando se desconoce del tema, si bien vimos un poco de estas en clase, es completamente diferente el manejar todo ya en físico, porque las conexiones varían al igual que el código, una de las mas grandes complicaciones que tuvimos fue el manejo de números flotantes o decimales, ya que al usar una configuración brindada por el profesor está si bien funcionaba solamente tomaba en cuenta 7 bits, dejando los ultimos 3 digitos generados como basura, si bien los mostraba estos o se redondean hacia arriba o los bajaba, al final para solucionar esto tuvimos que implementar la técnica de multiplicar todo x100 para “burlar” al microcontrolador y al final juntar número entero, el punto y el residuo para el decimal.

Referencias

[1] Microchip Technology. (2018). ATmega328P Datasheet (Rev. 7810D–AVR–01/15). Recuperado de https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

[2] Hitachi Semiconductor. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver. Recuperado de https://www.sparkfun.com/datasheets/LCD/HD44780.pdf

[3] Florgan, M. (2022). Embedded C Programming with the AVR Microcontroller. Packt Publishing.

[4] avr-libc Documentation. (2023). The avr-libc Reference Manual. Recuperado de https://www.nongnu.org/avr-libc/user-manual/